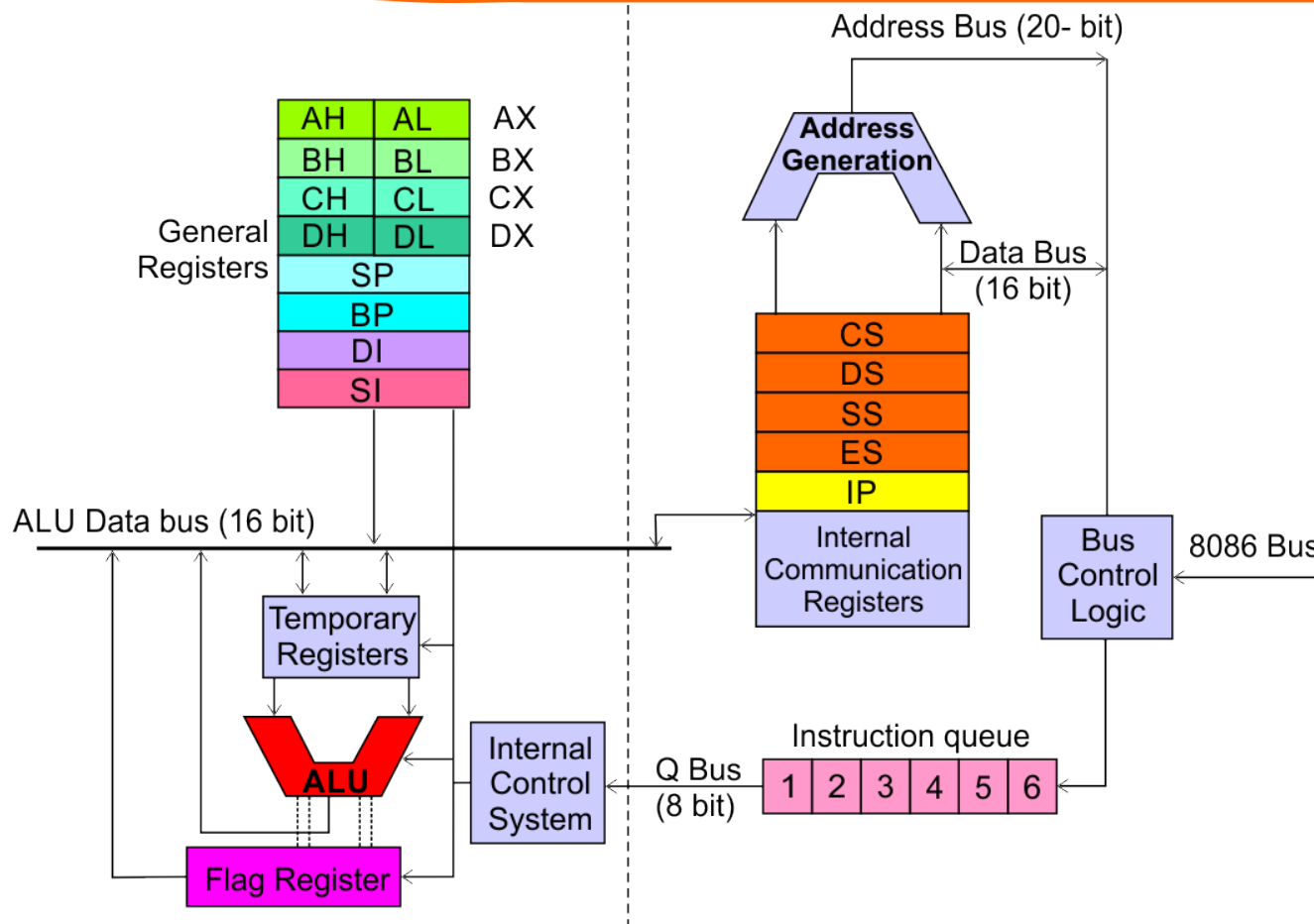


Architecture

Architecture



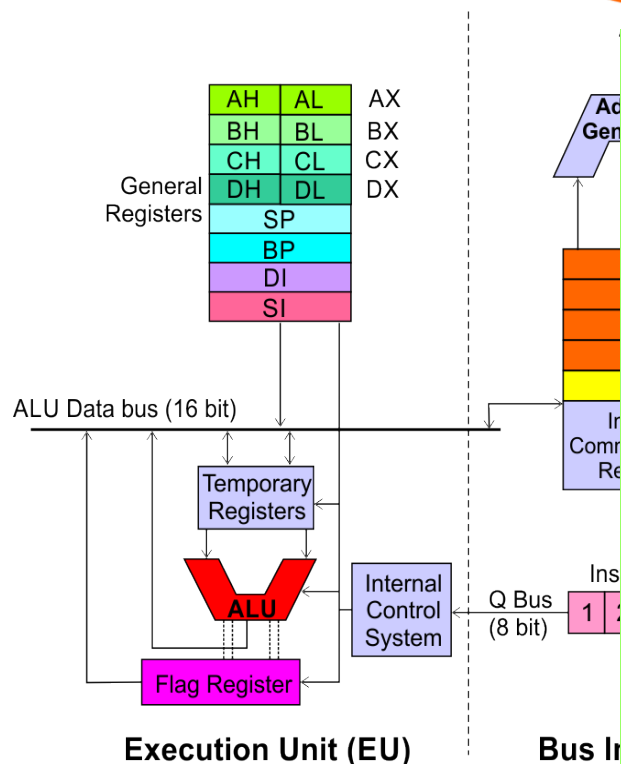
Execution Unit (EU)

EU executes instructions that have already been fetched by the BIU.

BIU and EU functions separately.

Bus Interface Unit (BIU)

BIU fetches instructions, reads data from memory and I/O ports, writes data to memory and I/O ports.



Execution Unit (EU)

• Main function

- **Control operations on data using the instruction decoder & ALU**
- **Gives instructions to BIU**
 - From where to fetch the data
 - **Decode and execute those instructions**
- **EU has no direct connection with system buses**
 - Shown in the figure
 - Performs operations over data through BIU

• Components

- **ALU**
 - It handles all arithmetic and logical operations
 - Example: +, -, ×, /, OR, AND, NOT
- **Flag Register**
 - A 16-bit register that behaves like a flip-flop
 - It changes its status according to the result stored in the accumulator
 - There are 9 flags, divided into 2 groups – Conditional Flags and Control Flags.

Conditional Flags

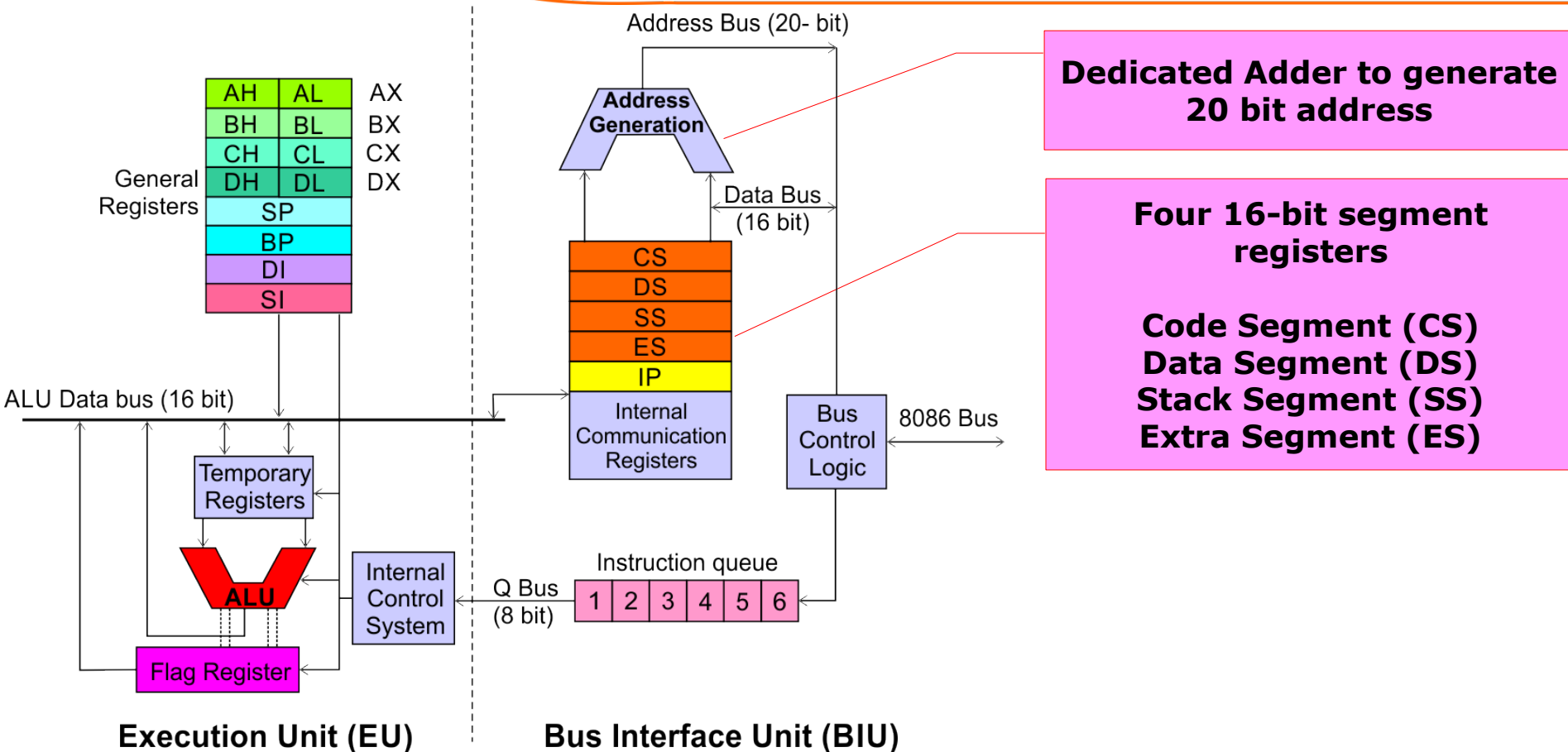
- **Carry flag** – It is used to show if a carry out of the most significant bit occurred during arithmetic op
- **Auxiliary flag** – The Auxiliary Carry Flag (AF) is a special flag used in microprocessors (like 8085 or 8086). It becomes 1 (set) if there is a carry from bit 3 to bit 4 during addition or a borrow from bit 4 to bit 3 during subtraction.
- **Parity flag** – This flag is used to indicate the parity of the result, i.e. when the lower order 8-bits of the result contains even number of 1s, then the Parity Flag is set. For odd number of 1s, the Parity Flag is reset
- **Zero flag** – This flag is set to 1 when the result of arithmetic or logical operation is zero else it is set to 0
- **Sign flag** – This flag holds the sign of the result, i.e. when the result of the operation is negative, then the sign flag is set to 1 else set to 0
- **Overflow flag** – This flag represents the result when the system capacity is exceeded

Control Flags

- **Trap flag** – It is used for single step control and allows the user to execute one instruction at a time for debugging. If it is set, then the program can be run in a single step mode
- **Interrupt flag** – It is an interrupt enable/disable flag, i.e. used to allow/prohibit the interruption of a program. It is set to 1 for interrupt enabled condition and set to 0 for interrupt disabled condition
- **Direction flag** – It is used in string operation. As the name suggests when it is set then string bytes are accessed from the higher memory address to the lower memory address and vice-versa

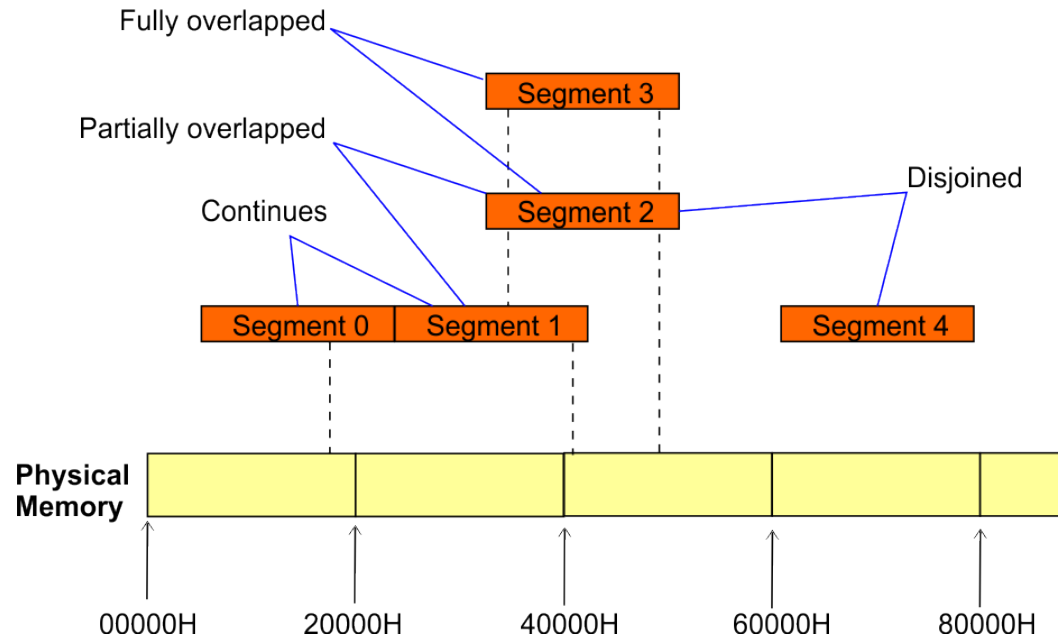
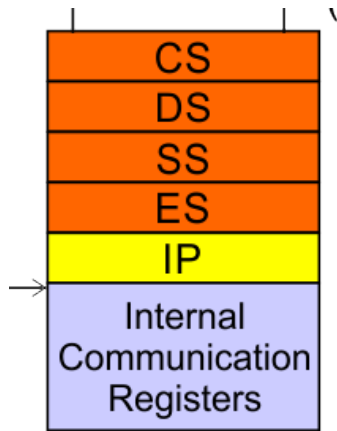
General purpose register

- **AX register** – It is also known as accumulator register. It is used to store operands for arithmetic operations.
- **BX register** – It is used as a base register. It is used to store the starting base address of the memory area within the data segment.
- **CX register** – It is referred to as counter. It is used in loop instruction to store the loop counter.
- **DX register** – This register is used to hold I/O port address for I/O instruction.
- **Stack pointer register** -It is a 16-bit register that holds the offset address of the top of the stack in the stack segment, and it is automatically updated during PUSH and POP operations.
- **Base Pointer(BP)** - It is used to store the base address of the memory.Used to access parameters or data on the Stack Segment (SS)
- **Source Index(SI)** - It is a memory pointer which is used to store the offset address of the source.
- **Destination Index(DI)** - It is a memory pointer which is used to store the offset address of the destination.



A segment register is a special-purpose register in the x86 microprocessor architecture that holds the starting address (base address) of a memory segment.

Segment Registers



- **8086's 1-megabyte memory is divided into segments of up to 64K bytes each.**

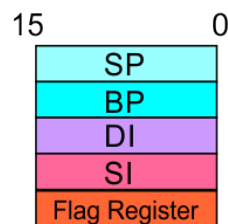
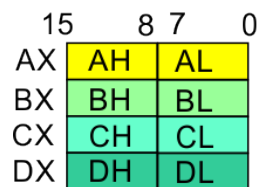
- **The 8086 can directly address four segments (256 K bytes within the 1 M byte of memory) at a particular time.**

- **Programs obtain access to code and data in the segments by changing the segment register content to point to the desired segments.**

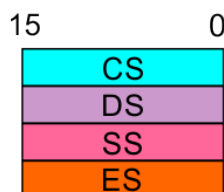
Segment Registers

Code Segment Register

- The Code Segment (CS) register is a 16-bit segment register in the 8086 that holds the starting address of the code segment in memory, and it is always used with the Instruction Pointer (IP) to access the next instruction to be executed
- BIU computes the 20-bit physical address by logically shifting the contents of CS 4-bits to the left and then adding the 16-bit contents of IP.
- That is, all instructions of a program are relative to the contents of the CS register multiplied by 16 and then offset is added provided by the IP.



EU



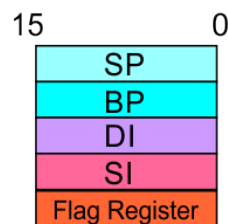
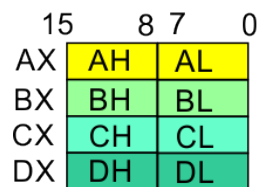
BIU

Segment Registers

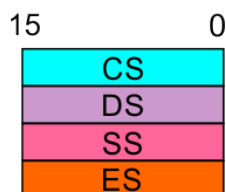
Data Segment Register

- 16-bit
- The Data Segment (DS) register points to the current data segment, where program data is stored, and operands for most instructions are fetched from this segment

The 16-bit contents of the Source Index (SI) or Destination Index (DI) or a 16-bit displacement are used as offset for computing the 20-bit physical address.



EU

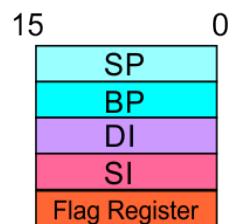
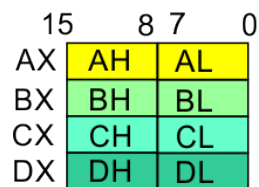


BIU

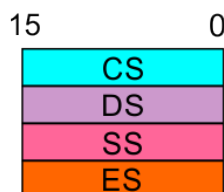
Segment Registers

Stack Segment Register

- The Stack Segment (SS) register is a 16-bit register that points to the current stack segment in memory, and it works with the Stack Pointer (SP) or Base Pointer (BP) to access data in the stack
- The 20-bit physical stack address is calculated from the Stack Segment (SS) and the Stack Pointer (SP) for stack instructions such as **PUSH** and **POP**.
- In based addressing mode, the 20-bit physical stack address is calculated from the Stack segment (SS) and the Base Pointer (BP).



EU

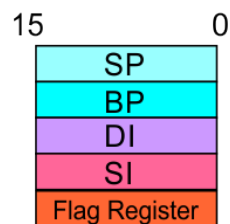
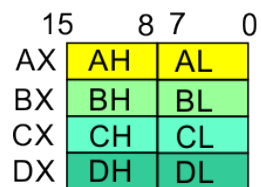


BIU

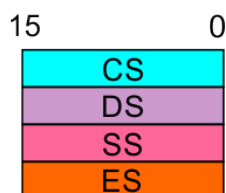
Segment Registers

Extra Segment Register

- The Extra Segment (ES) register is a 16-bit segment register that holds the base address of the extra data segment, and it is mainly used with the Source Index (SI) or Destination Index (DI) in string and memory operations . ES allows access to additional data beyond that 64KB.
- String instructions use the ES and DI to determine the 20- bit physical address for the destination.



EU

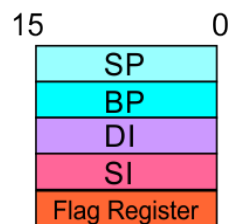
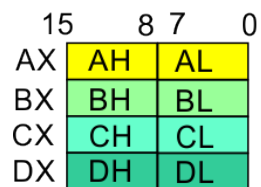


BIU

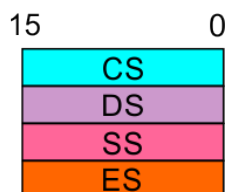
Segment Registers

Instruction Pointer

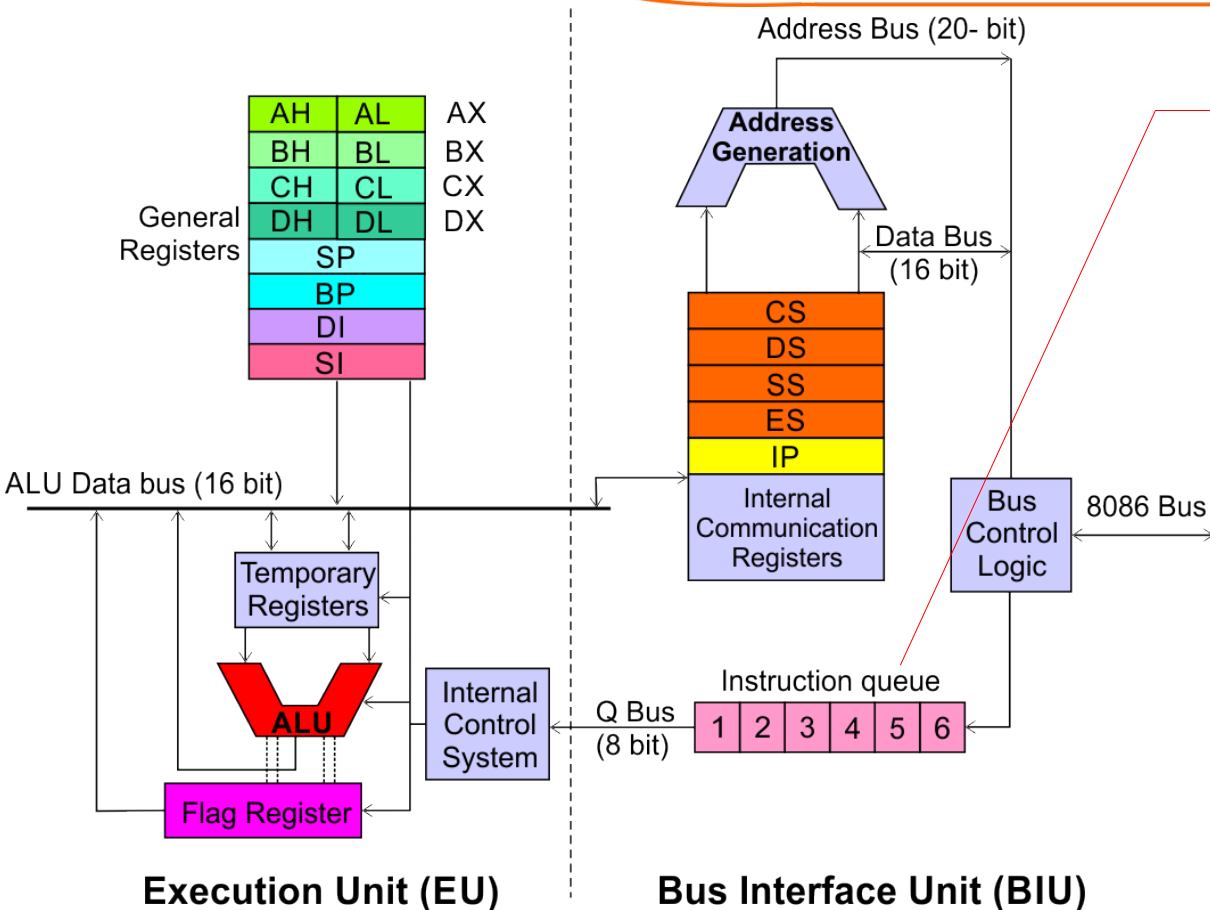
- 16-bit
- Always points to the next instruction to be executed within the currently executing code segment.
- So, this register contains the 16-bit offset address pointing to the next instruction code within the 64Kb of the code segment area.
- Its content is automatically incremented as the execution of the next instruction takes place.



EU



BIU



Instruction queue

- A group of First-In-First-Out (FIFO) in which up to 6 bytes of instruction code are pre fetched from the memory ahead of time.
- This is done in order to speed up the execution by overlapping instruction fetch with execution.
- This mechanism is known as pipelining.

The Instruction Queue in the 8086 is a 6-byte prefetch queue that stores instructions fetched from memory before they are executed, allowing the processor to overlap fetching and execution for faster processing.

EU decodes and executes instructions.

A decoder in the EU control system translates instructions.

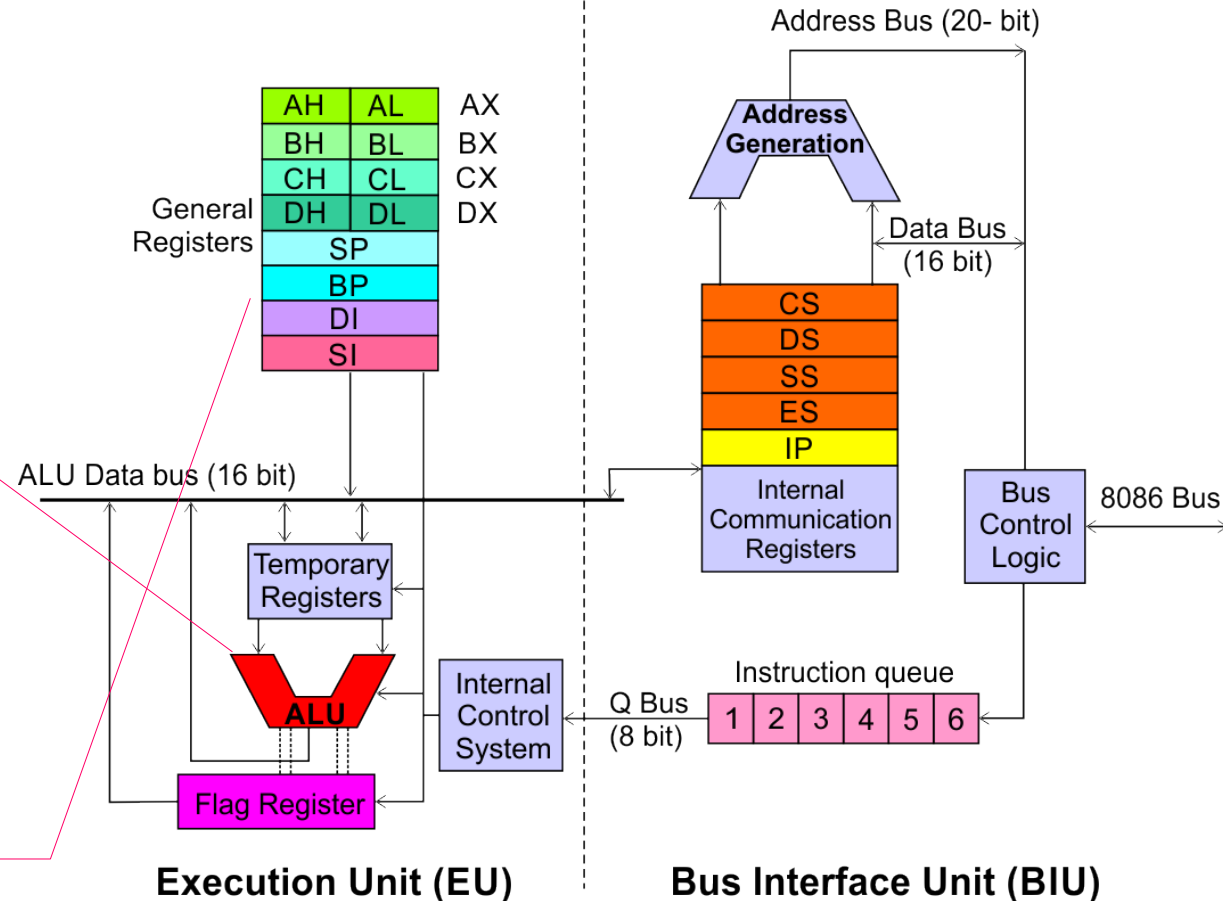
16-bit ALU for performing arithmetic and logic operation

Four general purpose registers (AX, BX, CX, DX);

Pointer registers (Stack Pointer, Base Pointer);

and

Index registers (Source Index, Destination Index) each of 16-bits



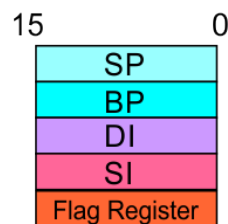
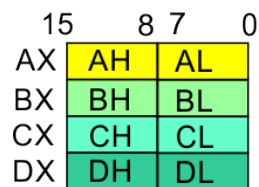
Some of the 16 bit registers can be used as two 8 bit registers as :

AX can be used as AH and AL
 BX can be used as BH and BL
 CX can be used as CH and CL
 DX can be used as DH and DL

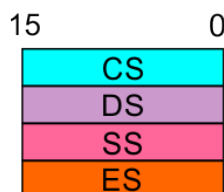
EU Registers

Accumulator Register (AX)

- Consists of two 8-bit registers AL and AH, which can be combined together and used as a 16-bit register AX.
- AL in this case contains the low order byte of the word, and AH contains the high-order byte.
- The I/O instructions use the AX or AL for inputting / outputting 16 or 8 bit data to or from an I/O port.
- Multiplication and Division instructions also use the AX or AL.



EU

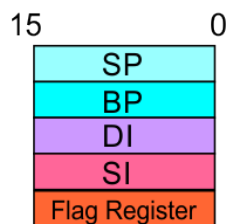
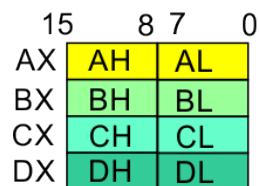


BIU

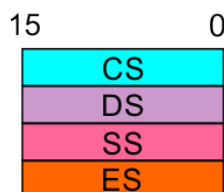
EU Registers

Base Register (BX)

- Consists of two 8-bit registers BL and BH, which can be combined together and used as a 16-bit register BX.
- BL in this case contains the low-order byte of the word, and BH contains the high-order byte.
- This is the only general purpose register whose contents can be used for addressing the 8086 memory.
- Whenever the processor uses this register to access memory, it will normally take the segment address from DS (Data Segment) register by default.



EU



BIU

EU Registers

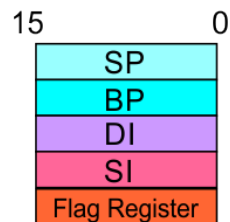
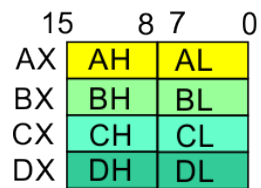
Counter Register (CX)

- Consists of two 8-bit registers CL and CH, which can be combined together and used as a 16-bit register CX.
- When combined, CL register contains the low order byte of the word, and CH contains the high-order byte.
- Instructions such as **SHIFT**, **ROTATE** and **LOOP** use the contents of CX as a counter.

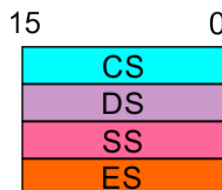
Example:

The instruction **LOOP START** automatically decrements CX by 1 without affecting flags and will check if [CX] = 0.

If it is zero, 8086 executes the next instruction; otherwise the 8086 branches to the label START.



EU

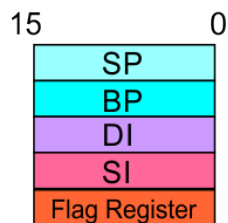
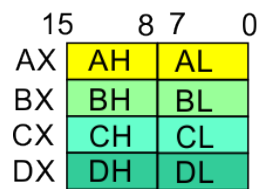


BIU

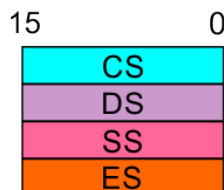
EU Registers

Data Register (DX)

- Consists of two 8-bit registers DL and DH, which can be combined together and used as a 16-bit register DX.
- When combined, DL register contains the low order byte of the word, and DH contains the high-order byte.
- Used to hold the high 16-bit result (data) in 16 X 16 multiplication or the high 16-bit dividend (data) before a $32 \div 16$ division and the 16-bit remainder after division.



EU



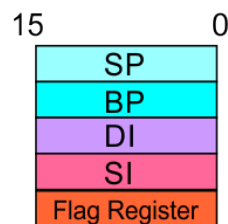
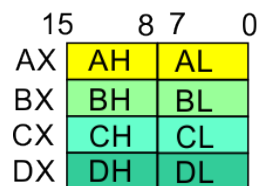
BIU

EU Registers

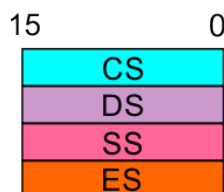
Stack Pointer (SP) and Base Pointer (BP)

- SP and BP are used to access data in the stack segment.
- The Stack Pointer (SP) is a 16-bit register used to keep track of the top of the stack. It works with the Stack Segment (SS) to point to where the last value was pushed.
- Whenever a PUSH instruction is used, SP is decreased (because the stack grows downward). When a POP instruction is used, SP is increased, as data is removed from the stack.

The Base Pointer (BP) is also a 16-bit register and is Used to hold the base value in base addressing using SS register to access data from stack memory.



EU

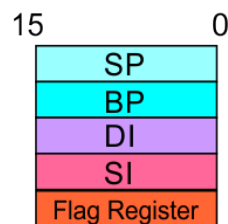
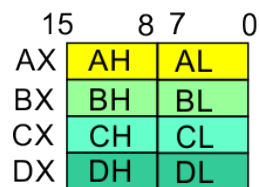


BIU

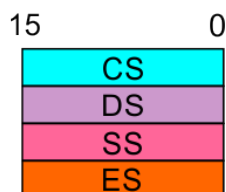
EU Registers

Source Index (SI) and Destination Index (DI)

- Used in indexed addressing.
- The Source Index (SI) and Destination Index (DI) are 16-bit registers used in indexed addressing, especially with string instructions. SI works with the DS (Data Segment) to point to the source data, while DI works with the ES (Extra Segment) to point to the destination. They help the CPU efficiently move, compare, or copy data blocks by automatically updating after each operation.



EU



BIU

Flag Register**Auxiliary Carry Flag**

This is set, if there is a carry from the lowest nibble, i.e, bit three during addition, or borrow for the lowest nibble, i.e, bit three, during subtraction.

Carry Flag

This flag is set, when there is a carry out of MSB in case of addition or a borrow in case of subtraction.

Sign Flag

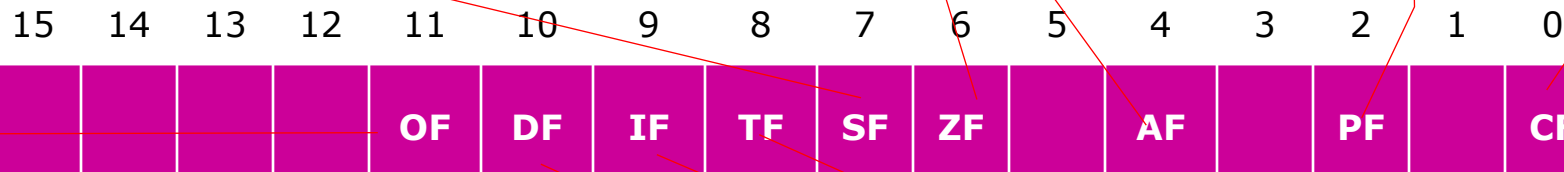
This flag is set, when the result of any computation is negative

Zero Flag

This flag is set, if the result of the computation or comparison performed by an instruction is zero

Parity Flag

This flag is set to 1, if the lower byte of the result contains even number of 1's ; for odd number of 1's set to zero.

**Over flow Flag**

This flag is set, if an overflow occurs, i.e, if the result of a signed operation is large enough to accommodate in a destination register. The result is of more than 7-bits in size in case of 8-bit signed operation and more than 15-bits in size in case of 16-bit sign operations, then the overflow will be set.

Tarp Flag

If this flag is set, the processor enters the single step execution mode by generating internal interrupts after the execution of each instruction

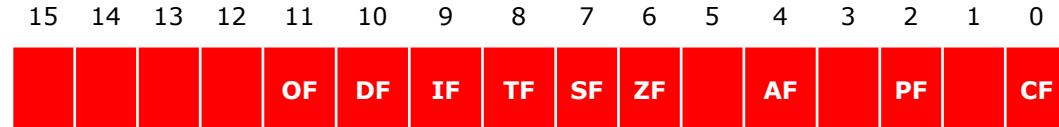
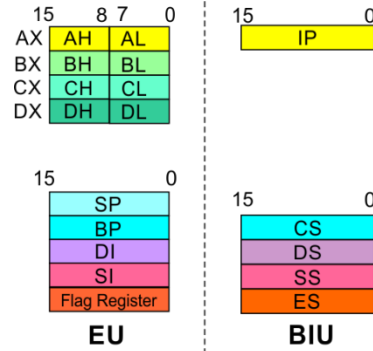
Direction Flag

This is used by string manipulation instructions. If this flag bit is '0', the string is processed beginning from the lowest address to the highest address, i.e., auto incrementing mode. Otherwise, the string is processed from the highest address towards the lowest address, i.e., auto decrementing mode.

Interrupt Flag

Causes the 8086 to recognize external mask interrupts; clearing IF disables these interrupts.

**8086 registers
categorized
into 4 groups**



Sl.No.	Type	Register width	Name of register
1	General purpose register	16 bit	AX, BX, CX, DX
		8 bit	AL, AH, BL, BH, CL, CH, DL, DH
2	Pointer register	16 bit	SP, BP
3	Index register	16 bit	SI, DI
4	Instruction Pointer	16 bit	IP
5	Segment register	16 bit	CS, DS, SS, ES
6	Flag (PSW)	16 bit	Flag register

Register	Name of the Register	Special Function
AX	16-bit Accumulator	Stores the 16-bit results of arithmetic and logic operations
AL	8-bit Accumulator	Stores the 8-bit results of arithmetic and logic operations
BX	Base register	Used to hold base value in base addressing mode to access memory data
CX	Count Register	Used to hold the count value in SHIFT, ROTATE and LOOP instructions
DX	Data Register	Used to hold data for multiplication and division operations
SP	Stack Pointer	Used to hold the offset address of top stack memory
BP	Base Pointer	Used to hold the base value in base addressing using SS register to access data from stack memory
SI	Source Index	Used to hold index value of source operand (data) for string instructions
DI	Destinaton Index	Used to hold the index value of destination operand (data) for string operations